NYCU-EE ICLAB – Spring 2026

Lab05 Exercise:

Design: Diffusion Model (DM)

Data Preparation

- Extract the test data from TA's directory:

  % tar xvf ~iclabTA01/LAB05_2026s.tar

- The extracted LAB directory contains:
  - Exercise/

Design Description

Diffusion Model (DM) is a generative model architecture designed to learn how to gradually remove noise from corrupted data and reconstruct the original sample. Unlike conventional discriminative models that directly map an input to a label or prediction, diffusion models learn the reverse process of a predefined noising procedure. By progressively denoising a noisy image step by step, the model can generate high-quality outputs and capture complex data distributions. For example, as illustrated in Fig. 1.

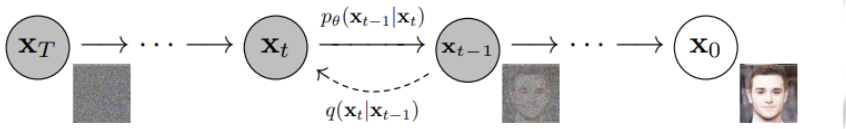


Fig. 1. The Diffusion Model example

In a traditional diffusion process, noise is gradually added to an input sample x_0 over multiple time steps until the data becomes nearly pure noise. The model is then trained to learn the reverse process, which **predicts and removes the added noise** at each step. If the noisy sample at time step t is denoted as x_t, the reverse process can be written as learning a function that estimates the noise component $\epsilon_\theta(x_t, t)$, so that the original sample can be progressively recovered. This procedure allows the model to generate or reconstruct images through iterative refinement, making diffusion models effective for image synthesis and restoration tasks.

In this exercise, you are required to design and implement a **simplified W4A8 Diffusion Model** architecture. Since the weights and activations require a large amount of storage, you are asked to use **memory (SRAM)** to complete this lab.

- Overall Architecture

The overall architecture of the Diffusion Model is illustrated in Fig. 2. The input is a **64 × 64 grayscale noisy image**.

First, the input image passes through a **down-sampling convolution layer**, which reduces the spatial resolution while extracting feature representations. The resulting feature map is then fed into a **Transformer block**, which consists of a **self-attention module** followed by a **feed-forward network**. The attention mechanism allows the model to capture global spatial relationships, while the feed-forward network further refines the extracted features. After the Transformer block, the feature map is processed by an **up-sampling convolution layer** to recover spatial information. Finally, an **interpolation operation** is applied to restore the feature map to the original resolution, producing the predicted noise component of the input image.

The denoising procedure is performed **iteratively for *k* iterations**, where the value of *k* is specified by the input. In **each iteration,** the model predicts the noise component from the current noisy image, and **the predicted noise is subtracted** from the input to obtain a cleaner image.

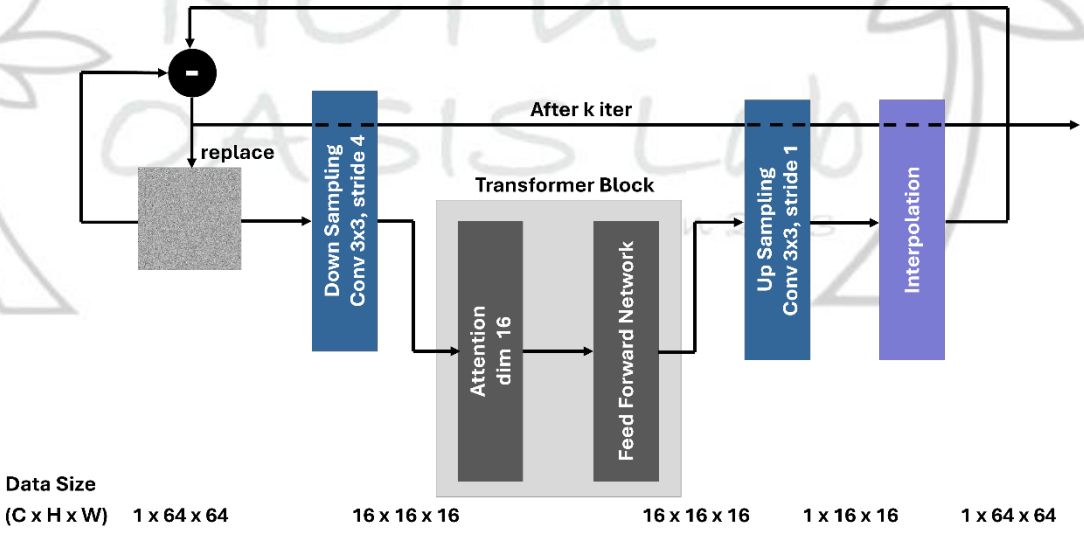


Fig. 2. Diffusion Model Architecture

- Weight

You will first receive the complete set of weights. The weights are input sequentially in the following order, and the sizes are given:

➢ **Down Sampling Conv: 3 × 3 × 1 × 16** (input channel → output channel) 0-143

➢ **Linear Transformation: 16 × 16 × 3 (q → k → v)** 0-47 144-911

144-159, 160-175, 176-191, 192-207, 208-223, ..., 896-911, 912 128 bit

128 bit : [0:7] → [120:127]

912 – 927 , 928 – 943 , 944 – 959 , 960 – 975 , 976 – 991 , ... , 1152 – 1167 , 1168

928 ÷ 16 = 58

128 bit

48 – 63

912 – 1169

- **FFN: 16 × 16**
- **Up Sampling Conv: 3 × 3 × 16 × 1** 144 – 287 1168 – 1311

In the **convolution layers**, the data is arranged according to one complete output channel at a time. As shown in Fig. 3, the nine values of a **single 3 × 3 kernel** are transmitted in **raster-scan order**. For the down sampling convolution, consecutive kernels correspond to different **output channels.** For the up sampling convolution, consecutive kernels correspond to different **input channels**.

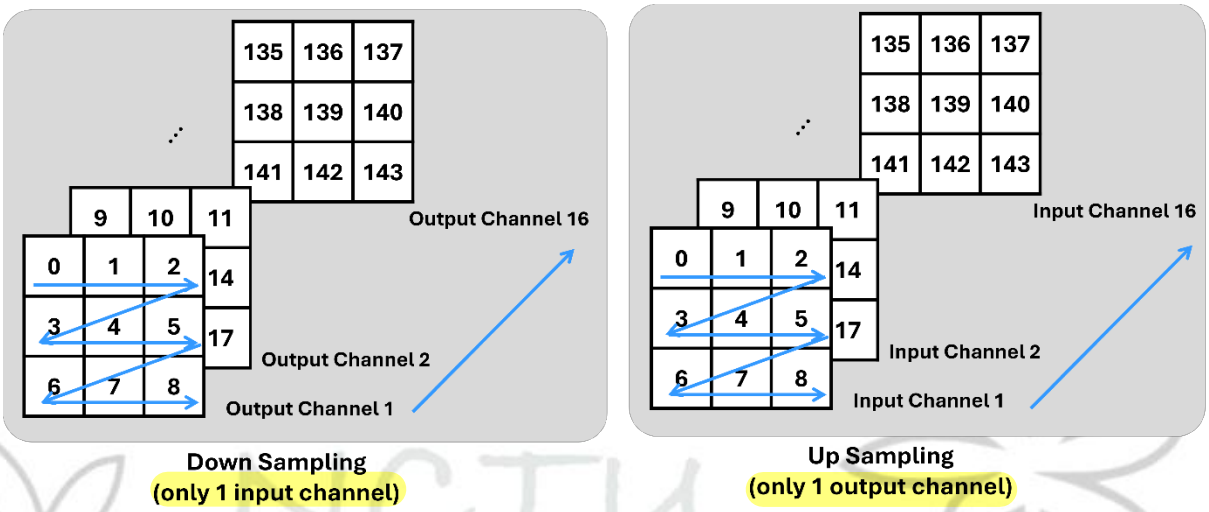


Fig. 3. Weight value of convolution layers

As shown in Fig.4, the weights of **the transformer block** are transmitted in **raster-scan order**. The weights are provided in the order of the **query (Q)**, **key (K)**, and **value (V)** projection matrices, followed by the weights of the **feed-forward network**.

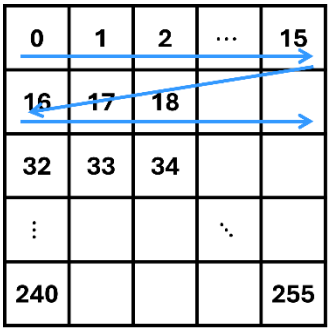


Fig. 4. Weight value of transformer block

Each weight is **4 bits signed** value and is provided through **i_weight.**

- Image

Next, you will receive the image input. The **image size** is **64 × 64**, and the pixel values are streamed in **raster scan order**.

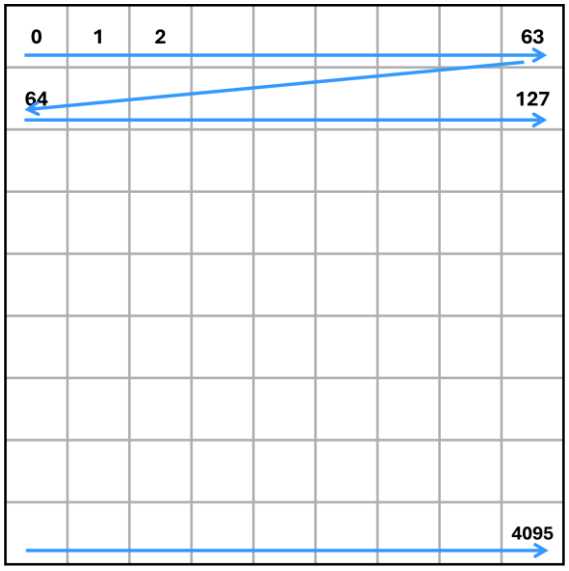


Fig. 5. Input image of size 64 × 64

Each weight is **8 bits unsigned value** and is provided through **i_data.**

After the final values for the denoised image are output, the next input image will be provided.

- Calculate instruction
  - Convolution with Different Input and Output Channels

    The computation method for convolution with **different numbers of input and output channels** is described as follows.

    For each output channel, there is a corresponding set of kernels. Each kernel is associated with one input channel. The convolution is performed by multiplying the kernel values with the corresponding spatial positions of the input feature maps **channel by channel**. The results from all input channels are then **summed together** to produce a single value in the output channel. The overall process is illustrated in Fig. 6.

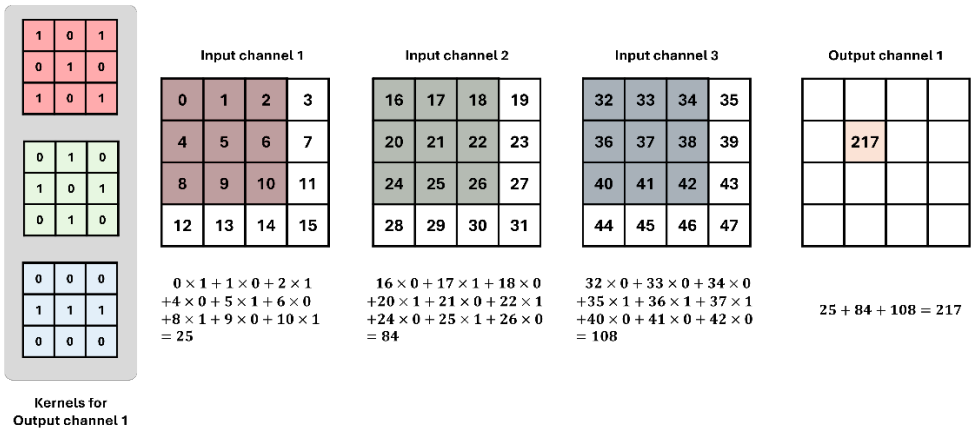


Fig. 6. Examples of 3 input channels, performing convolution with 3x3 kernel and stride 1

➢ Convolution Layer

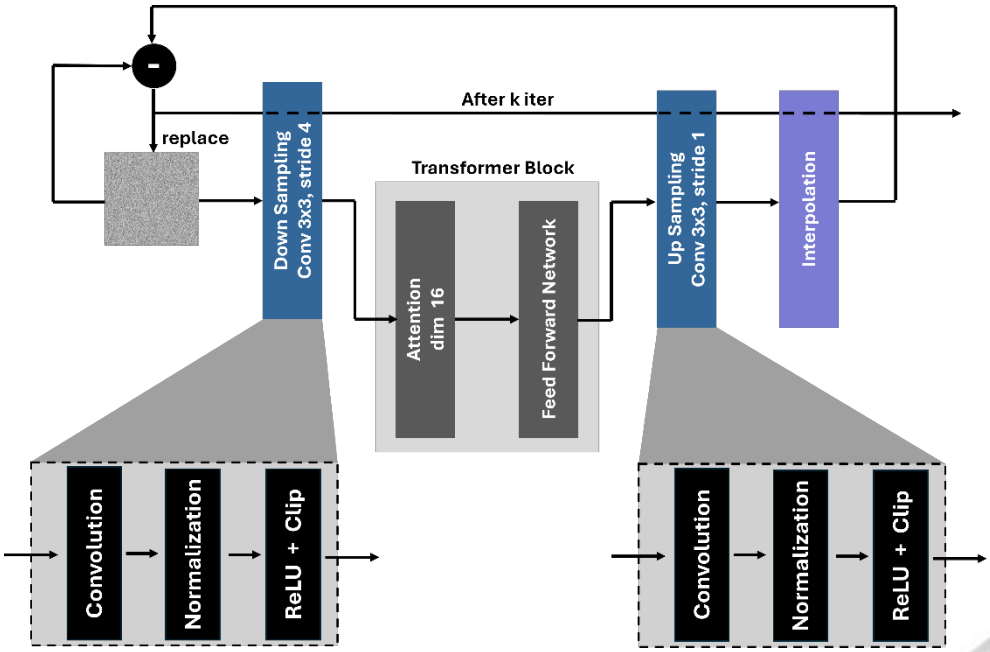


Fig. 7. Convolution process flow

Fig. 7 illustrates the processing flow of the **convolution layer.** Except for the differences in the convolution operation itself, the **up sampling convolution** and **down sampling convolution** follow the same processing procedure.

For the **down sampling convolution**, a convolution with **stride 4**, a **3 × 3 kernel**, and **zero padding** is applied. In this stage, **one input channel is expanded into 16 output channels**.

For the **up sampling convolution**, a convolution with **stride 1**, a **3 × 3 kernel**, and **zero padding** is used. In this stage, **16 input channels are combined into one output channel**.

After the convolution operation, both modules are followed by **batch normalization**. A **ReLU activation function** is then applied to **clip the output values within the range of [0, 255]**.

◆ Convolution

The **stride 4 convolution** is shown in Fig. 8. In this operation, the convolution kernel moves **four steps at a time**. As a result, the spatial dimensions of the output feature map are reduced to **one quarter** of the original input size.

Fig. 9 is the **stride 1 convolution**. In this operation, the convolution kernel moves **one step at a time**. As a result, the spatial dimensions of the output feature map are **the same** as the original input size.

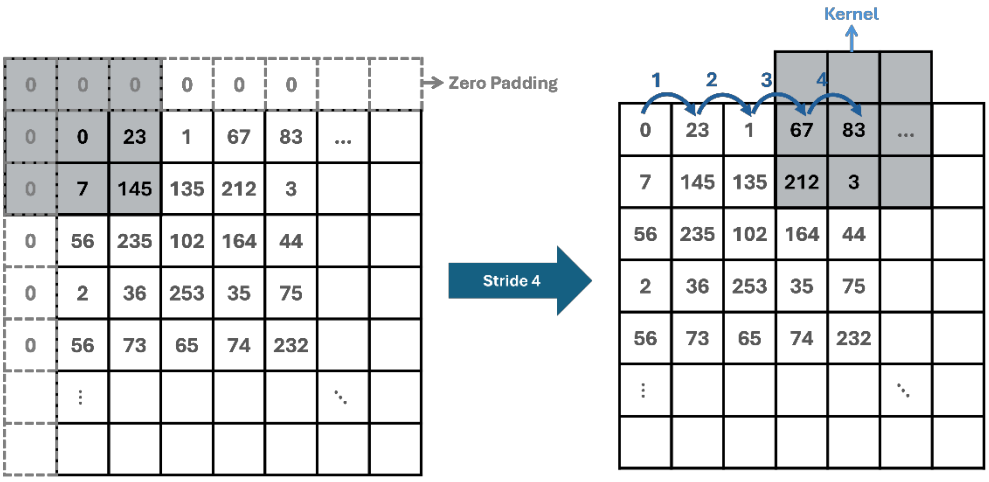


Fig. 8. Convolution with 3x3 kernel, **stride 4**, zero padding

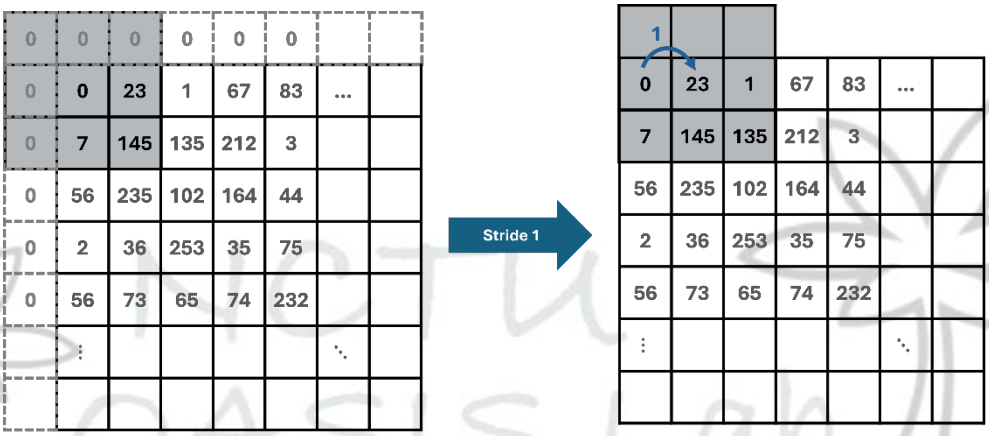


Fig. 9. Convolution with 3x3 kernel, **stride 1**, zero padding

◆ Normalization

After applying **batch normalization fusion**, the batch normalization operation can be simplified as

$$\textbf{Batch norm result} = \frac{\textbf{CNN output value}}{a}$$

In this design, the constant is fixed as $a = 2^6$ for both convolution layers. Therefore, the computation can be simplified to a **right shift operation**:

$$\textbf{Batch norm result} = \textbf{CNN output value} \ggg 6$$

After the shift operation, a **midpoint offset** is added before applying the activation function. The midpoint for both convolution layers is **set to 128**:

$$\textbf{Shifted value} = (\textbf{CNN output value} \ggg 6) + 128$$

◆ ReLu and Clipping

Finally, **ReLU and clipping** are applied to ensure the activation values remain within the valid **8-bit range [0, 255]**, where values less than 0 are set to 0, values between 0 and 255 remain unchanged, and values greater than 255 are clipped to 255.

128 bit addr : 64 – 319

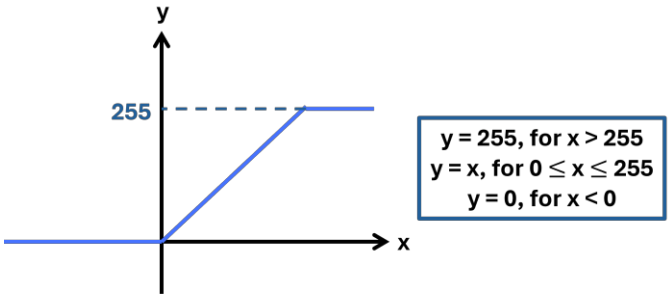


Fig. 10. ReLU + Clipping

➢ Feature Map Reordering before the Transformer Block

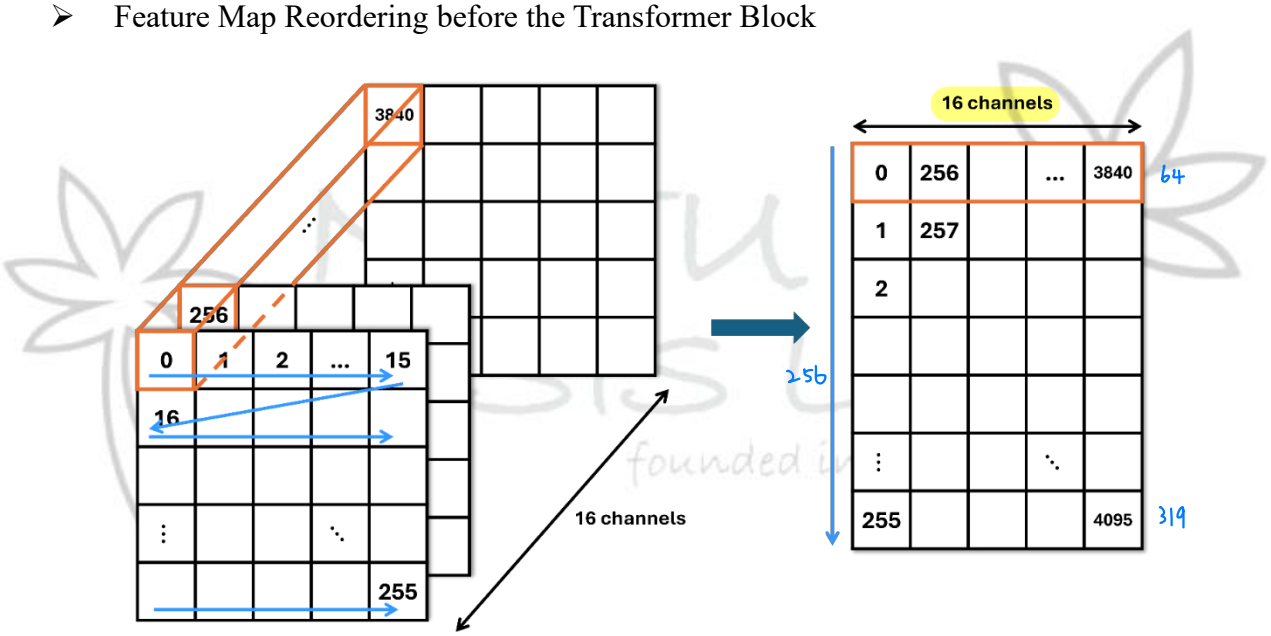


Fig. 11. Feature Map reordering process

As shown in Fig. 11, before entering the Transformer block, the output of the convolution layer must be **reordered to form a matrix** suitable for the Transformer operations. This reordering process groups the **values of the same pixel location across different channels** into a single feature vector. In other words, each feature point consists of a feature vector with a **length of 16,** corresponding to the **16 channels**.

Since the feature map size is 16 × 16, there are a total of **256 (16 × 16) feature points**. These feature vectors are then arranged sequentially in **raster scan order.** After the reordering process, the data is reshaped into a **256 × 16 matrix,** which is then used as the input to the Transformer block for attention and FFN computations.

➢ Transformer Block

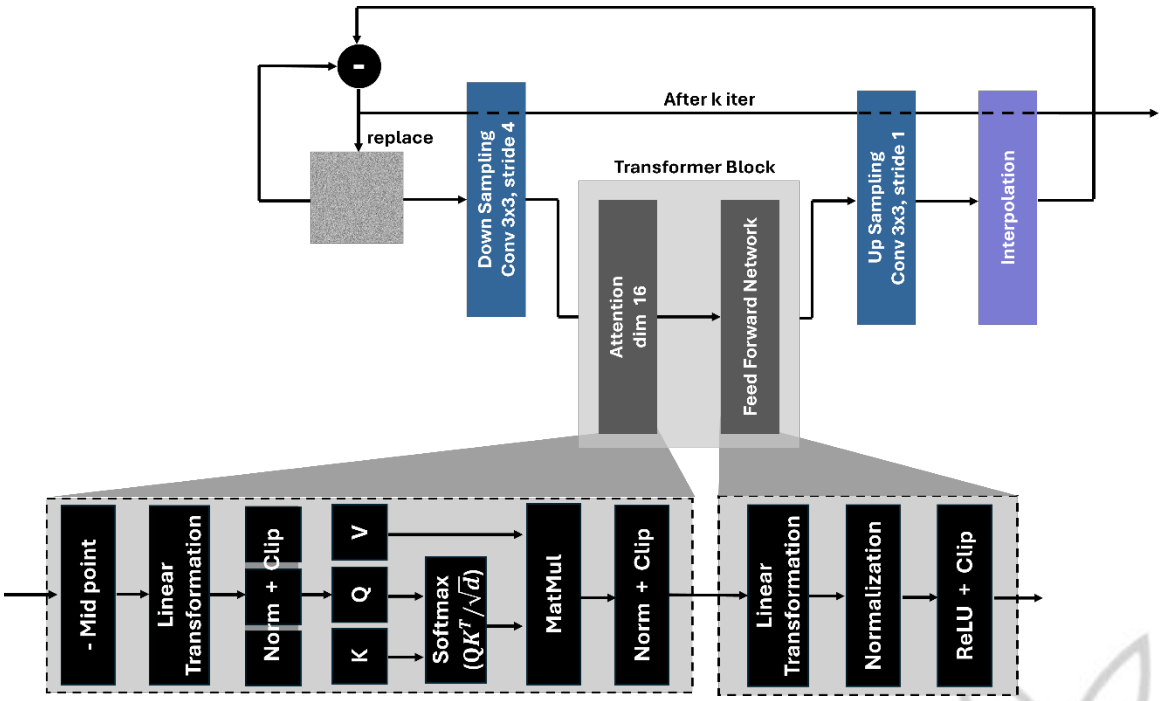


Fig. 12. Transformer Block process flow

Fig. 12 illustrates the processing flow of the **Transformer block**, which consists of two main parts: the **Attention module** and the **Feed-Forward Network**.

Before starting the attention computation, the input matrix must first **subtract the midpoint value (128)** in order to convert the **midpoint of the data back into 0** for subsequent calculations.

signed

$$\textbf{calc value} = \textbf{conv output} - 128$$

Next, the input matrix undergoes **linear transformations** with different weight matrices to generate the **Query (Q)**, **Key (K)**, and **Value (V)** matrices. After these matrices are obtained, **normalization and clipping** are applied to convert the results back to **8-bit signed values** before proceeding to the following computations.

The attention function is shown as follows:

$$\text{Attn} = \text{SoftMax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

The **Query matrix (Q)** and the **transpose of the Key matrix (K^T)** are then multiplied to produce an intermediate result. Afterward, the **SoftMax operation** is applied. The resulting matrix is then multiplied by the **Value matrix (V)** to produce the final output of the attention module. The result is again **normalized and clipped to 8-bit signed values**, producing the **attention output**. The detailed

computation of these operations will be explained in the following section.

In the **Feed-Forward Network (FFN)** stage, the computation mainly consists of a **single linear transformation**. After this transformation, **normalization, ReLU, and clipping** are applied to obtain the final output of the **Transformer block**.

- Linear Transformation

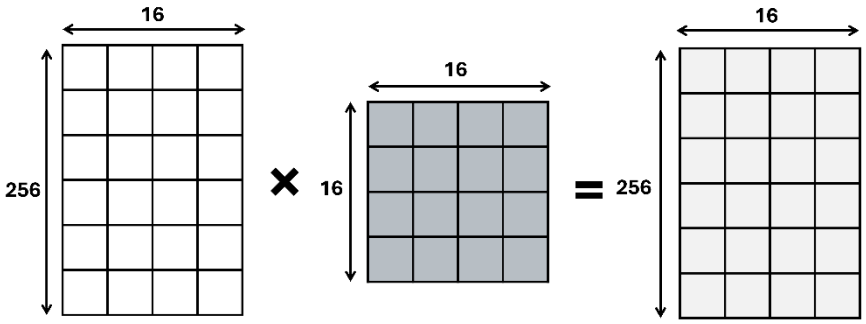


Fig. 13. Linear transformation

The linear transformations used to generate **Q, K, and V**, as well as the linear transformation in the **Feed-Forward Network**, are illustrated in Fig. 13.

The input is a **256 × 16 matrix**, which is multiplied by a **16 × 16 weight matrix** through **matrix multiplication**. The resulting output is a **256 × 16 matrix**. This operation performs a **linear projection** on each feature vector while preserving the number of feature points.

In this design, the **normalization applied after all linear transformation** is fixed with a constant $a = 2^4$. Therefore, the normalization operation can be simplified as a **right shift operation**:

$$\textbf{norm result} = \textbf{linear transformation output} \ggg 4$$

This operation scales the output of the linear transformation before it is clipped to the valid **8-bit range** for subsequent computations.

For the **Q, K, and V projections**, the normalized results are **directly clipped to the signed 8-bit range** $[-128, 127]$.

For the **Feed-Forward Network (FFN)**, the post-processing follows the same procedure as the **CNN layers**. First, a **midpoint offset of 128** is added to the normalized result. Afterward, **ReLU and clipping** are applied, which constrain the output values to the range **[0, 255]**.

◆ Attention

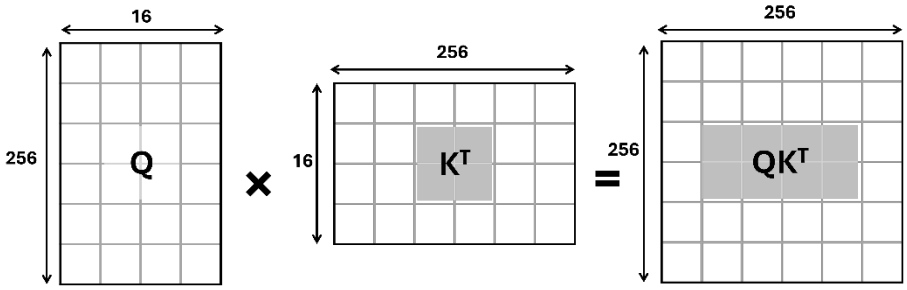


Fig. 14. Calculation process

As shown in Fig. 14, after multiplying **Q** with the transpose of **K**, a **256 × 256 matrix** is obtained. At this stage, the computation of

$$\text{SoftMax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

should be performed. Since $\mathbf{d = 16}$ the result is first **divided by 4**, here you can directly round down the value. >>> 2

Next, the **SoftMax operation** is applied. To simplify the computation, the SoftMax function is approximated as a **piecewise linear operation,** as shown in Fig. 15. Specifically, values within the range **[-2048, 2047]** are mapped into the range **[0, 1]**.

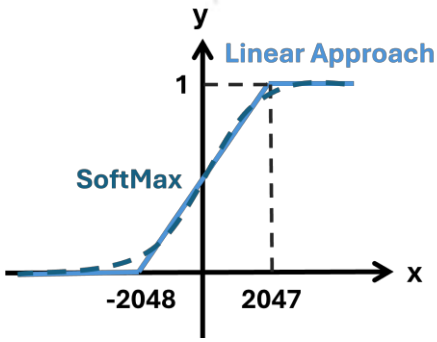


Fig. 15. Linear Approach of SoftMax function

12 bit

First, the values are **clipped to the range [-2048, 2047]**. Then, they are converted into **0.8 fixed-point format** using the following transformation:

$$\mathbf{fixpoint = max\,(0,\ min\,(255,\ (clip\ point \ggg 4) + 128))}$$

The resulting **fixed-point values** are then used for the subsequent computation by

performing **matrix multiplication with the Value matrix (V)**. This operation produces a **256 × 16 output matrix**.

After the matrix multiplication, the result is converted back to a **signed 8-bit integer value** using the following operation:

$$\text{output} = \min(127, \max(-128, \text{output matrix} \ggg 16))$$

The right shift operation compensates for the scaling introduced by the fixed-point representation during the matrix multiplication.

- Feature Map Reordering after the Transformer Block

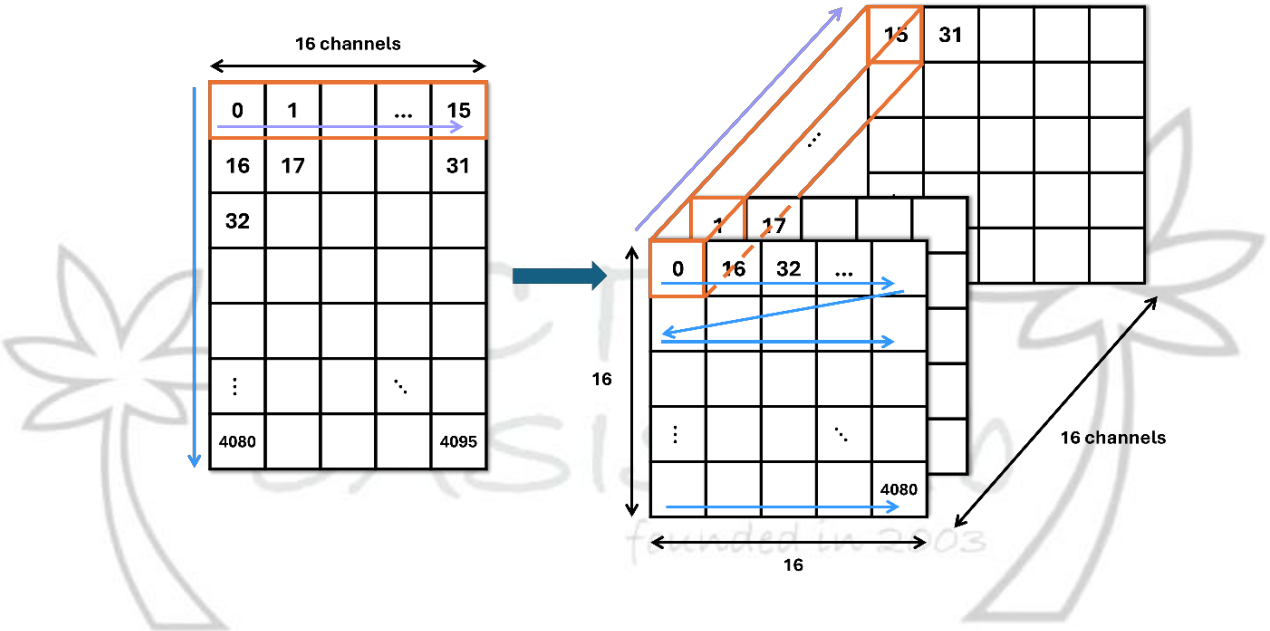


Fig. 16. Reordering process before convolution calculation

As shown in Fig. 16, after the **Transformer block**, the output matrix must be **reshaped back to the 16 × 16 × 16 spatial feature map**.

The Transformer output is a **256 × 16 matrix**, where each row represents the **feature vector of a feature point** with a length of **16**. For each feature point, the **16 elements of the feature vector** are assigned to the **16 channels** of that spatial location. After this reordering process, the data is reshaped into a **16 × 16 × 16 feature map**, which can then be used as the input to the **convolution layer** for subsequent processing.

➢ Interpolation and Denoise

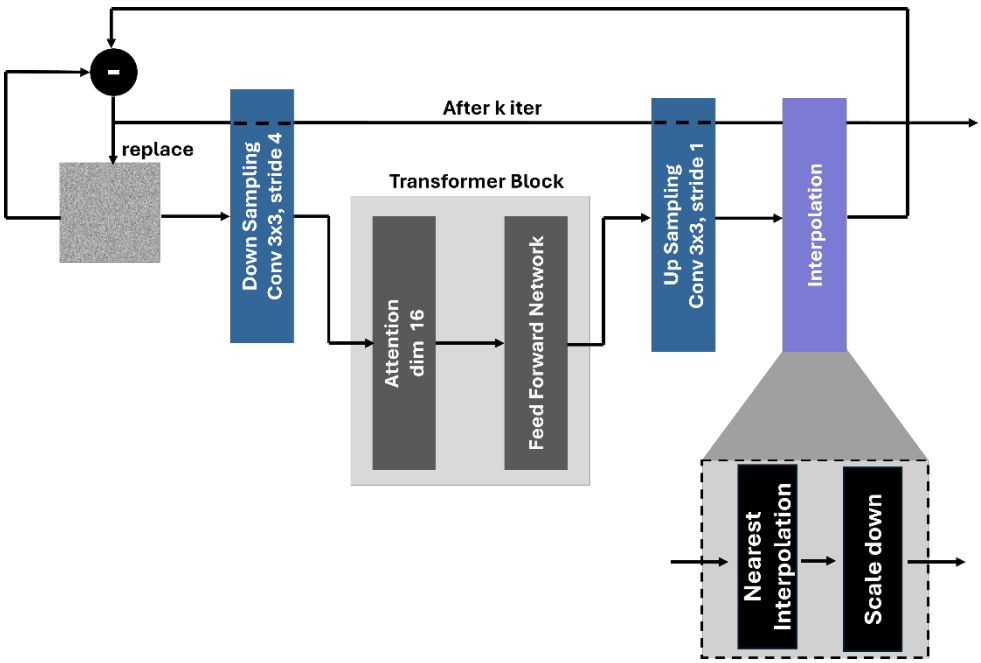


Fig. 17. Interpolation process

Fig. 17 illustrates the processing flow of the **interpolation** stage. In this stage, the **16 × 16** convolution output is upscaled back to a **64 × 64** image. **Three interpolation modes** are supported in this design. The specific processes are shown in Figs. 18 to 20.

Mode 0 uses **nearest-neighbor interpolation**, which **replicating each pixel into a 4 × 4 block.**

Mode 1 performs **two-point interpolation** along the **x-direction**, where each row is expanded horizontally based on adjacent pixels; for the **rightmost boundary**, **extrapolation** is applied using the **pixel on the left**.

Mode 2 performs **two-point interpolation** along the **y-direction**, where each column is expanded vertically based on adjacent pixels; for the **bottom boundary**, **extrapolation** is applied using the pixel from the **previous row**.

For both **Mode 1** and **Mode 2**, to reduce **the division error** in hardware implementation, **the difference** between two neighboring points is **first calculated** and scaled by a **shift operation**. The interpolation result is then generated by **accumulating the difference** with the reference point. If the **extrapolation** caused the value to **fall outside [0, 255]**, **clip** it back to [0, 255].

1f = 0001 1111

16+8+4+2+1 = 31

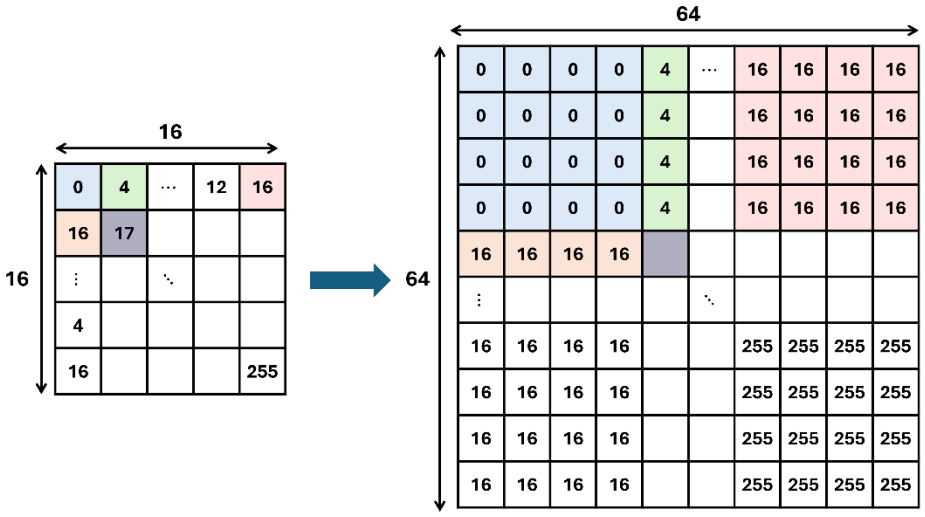


Fig. 18. Mode 0 interpolation

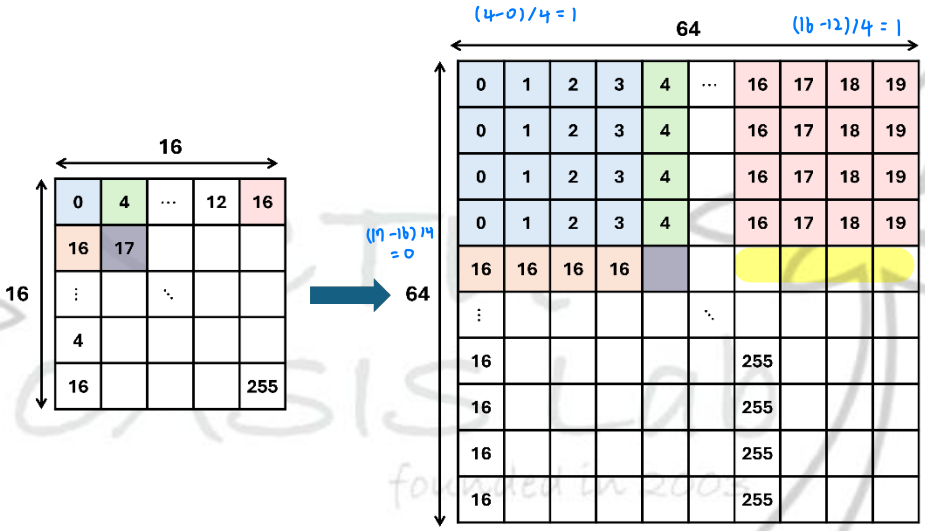


Fig. 19. Mode 1 interpolation

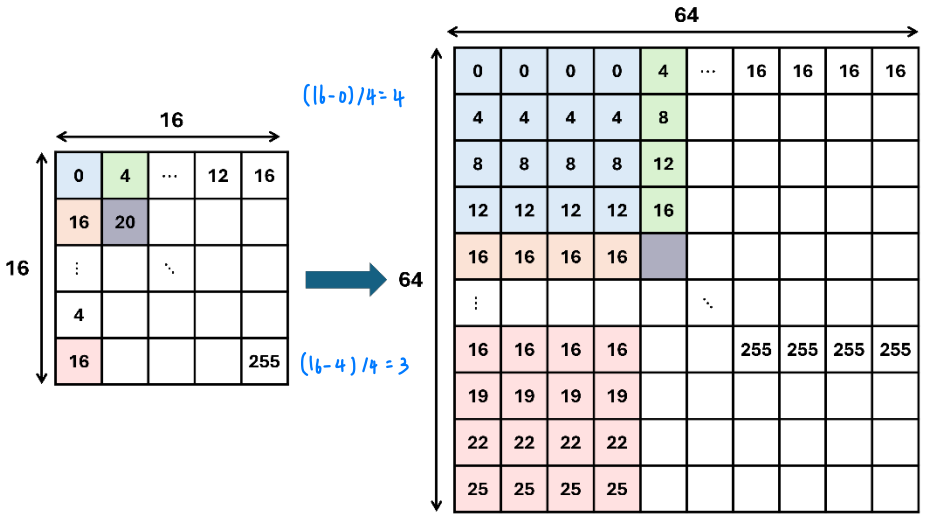


Fig. 20. Mode 2 interpolation

After the interpolation process, the predicted **noise value** is scaled down. This is implemented by performing a **right shift of 3 bits**, producing a **5-bit noise value**.

The input image of the current iteration is then **subtracted by the scaled noise value** to obtain the denoised result. The result is clipped to ensure that all pixel values remain within the valid **[0, 255]** range. The operation can be expressed as follows:

$$\textbf{denoise image} = \min\left(255, \max\left(0, \textbf{input image} - (\textbf{interpolation output} \gg 3)\right)\right)$$

This denoised image is then used as the **input for the next iteration** of the diffusion process, or the **output of the last iteration**.

I/O Description

Input	Bit Width	Definition and Description
clk	1	Clock
rst_n	1	Asynchronous active-low reset
i_valid	1	High when sending **weight/image/iteration/mode** values.
i_iter	3	The number of **denoising iterations** for the input image is transmitted through **i_iter** and is valid only in the **first cycle** of the input sequence. The iteration value ranges from **1 to 7**. **One iteration** refers to performing the **complete denoising pipeline once**.
i_mode	2	The mode of **interpolation** for the input image is transmitted through **i_mode** and is valid only in the **first cycle** of the input sequence.
i_data	8	When "i_valid" is high, i_data is sending the **image data** in **raster scan order**, where it contains 8 bits unsigned integer value.
i_weight	4	When "i_valid" is high, i_weight sends the value of **weight**.
Output	**Bit Width**	**Definition and Description**
o_valid	1	High only when **o_data** is valid. It cannot be overlapped with **i_valid** .
o_data	8	It will output the final result of the diffusion model in **in raster scan order** after finishing the calculation. Each **o_data** contains 8 bits unsigned value for one denoised pixel value. Output doesn't need to be continuous.

Specification

Top module

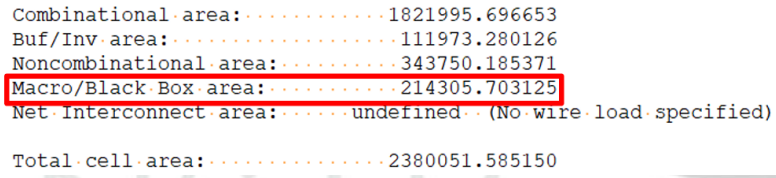
1. Top module name: **DM** (file name: **DM.v**)

2. You can adjust your clock period by yourself, but the maximum period is 20 ns. The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.

3. The execution latency of each iteration is limited to 150,000 cycles. The latency is measured starting from the falling edge of the last cycle of i_valid until all denoised image outputs are completed. During this interval, only cycles where o_valid is low are counted toward the latency. If the o_valid pulls high after the falling edge of i_valid, the latency is counted as 1 cycle.

4. The total cell area should not larger than 9,000,000 μm^2.

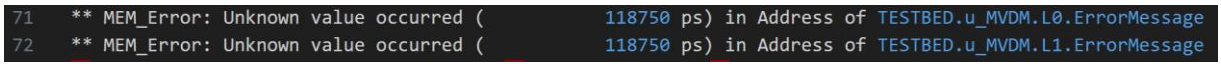
5. In this lab, you must use the memory and generate it yourself. The number of words and the bits per each word is defined by yourself. The total number and kind of memory is unlimited. We will check it at **DM.area in 02_SYN/Report**. The area of **Macro/Black Box** must not be 0. The example is shown in following figure.



6. For the register usage, the non-combinational area must not exceed 1,000,000 μm^2. This constraint is intended to prevent storing all activations in registers for computation.

7. Baseline: Area < 9,000,000 μm^2, Register Area < 1,000,000 μm^2, Average latency < 3,000,000 ns for 1 iteration (average).
   Goal: Average latency < 800,000 ns, Performance < 8.0×10^{18} for 1 iteration (average).
   You cannot get a full score if you only pass the baseline.

8. There should be no MEM_error allowed in the simulation, and when having memory errors, you will fail this lab.



9. If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message. Even though the design may still pass gate level simulation, this situation will be regarded as a synthesis fail. In this case, the memory area will be 0 in **DM.area**. We will check it at **syn.log** and **DM.area**.

Reset

10. It is asynchronous reset and active-low architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.

11. The reset signal (**rst_n)** will be given only once at the beginning of simulation. All output signals should be reset to 0 after the reset signal is asserted.

i_valid & input signals

12. All input signals are synchronized at negative edge of the clock.

13. **i_valid** will come after reset.

14. **Input data is sent by i_weight for weights, and i_data for the noisy image.**

15. The **weights** for **convolution** are delivered output channel by output channel. For each output channel, the kernel values of one input channel are provided in raster scan order, covering all input channels corresponding to that output channel. The **weights** for the **transformer block** are input in raster scan order. The input will be delivered continuously for 1312 cycles when **i_valid** is tied high. When all elements of the weights are delivered completely, it is tied to unknown state, and **i_valid** will also be tied low.

16. The **weights** are provided only at the beginning and must be reused for all subsequent image inputs.

17. The input of **image** is delivered in raster scan order for 64x64 cycles continuously when **i_valid** is tied high. When all elements of the images are delivered completely, it is tied to unknown state, and **i_valid** will also be tied low.

18. After input of weight, the first **i_valid** of image will come in 1~3 negative edge of clock after **i_valid for weight** falls.

o_valid & o_data

19. All output signals should be synchronized at clock positive edge.

20. The TA's pattern will capture your output for checking at clock negative edge.

21. **o_valid** cannot overlap with **i_valid** at any time.

22. **o_valid** is limited to be high only when **o_data** is valid.

23. **o_data** should be correct when **o_valid** is high. And **o_data** should be low when **o_valid** is low.

24. **o_data** must be delivered for a total of 64x64 cycles, and o_valid needs to be high simultaneously.

25. The output doesn't need to be continuous.

26. The next input pattern will come in 1~3 negative edge of clock after the **o_valid** of this pattern falls.

Synthesis

27. The input delay is set to 0.5*(clock period).

28. The output delay is set to 0.5*(clock period), and the output loading is set to 0.05**.**

29. The input delay of **clk** and **rst_n** should be zero.

30. The synthesis time should be less than 2 hour.

31. Do not write comment in the filelist.f file, or the simulation may pass in your terminal but will FAIL in TA's.

```
1  TESTBED.v
2  //../04_MEM/L0_16index_ver_1024X128.v
3  //../04_MEM/L0_index8_2048X64.v
4  ../04_MEM/L0_index16_1024X128_SP.v
5  ../04_MEM/L1_index16_1024X128_SP.v
```

(NEVER do that)

32. The synthesis result (syn.log) of data type cannot include any latches and error.

33. After synthesis, you can check **DM.area** and **DM.timing**. The area report is valid when the slack in the end of timing report should be non-negative and the result should be MET.

Gate-level simulation

34. The gate-level simulation cannot include any timing violations without the *notimingcheck* command.

Supplement

35. In this lab, you are NOT allowed to use Designware IP.

36. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

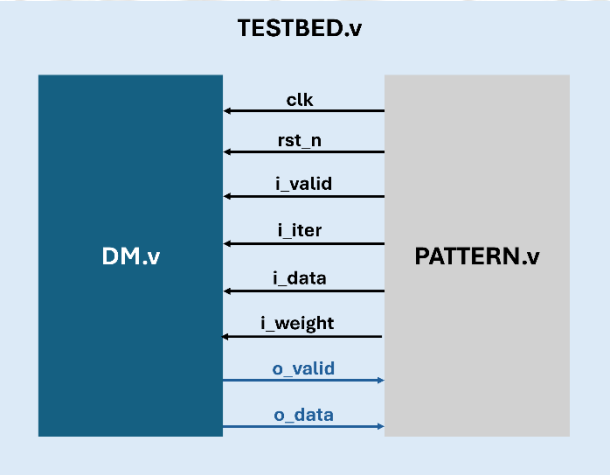
37. Don't write Chinese or other language comments in the file you sent.

38. Verilog command //synopsys dc_script_begin, //synopsys dc_script_end //synopsys translate_off, //synopsys translate_on are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.

39. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.

40. Any form of display or printing information in verilog design is forbidden. You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your design. The less cost your design has, the higher grade you get.

- Function Validity: 70%
- Goal: You need to pass the **goal** to get the Performance score

- Performance: 30%
  - Performance = Total Area * Non-Comb Area * Total latency
  - Total latency = cycle time * latency(1 iteration).
- Performance Comparison (1de vs. 2de):
  - 1de and 2de submissions will be evaluated and ranked separately.
  - The total score for 1de submissions ranges from 70 to 100.
  - The total score for 2de submissions ranges from 49 to 70.

Note

- Please submit your design (DM.v) in LAB05_26s /EXERCISE/09_SUBMIT
  - 1st_demo deadline : 2026/04/06 (Mon.) 21:00:00

    Due to the power outage at Engineering Building IV from 0:00 AM on April 4, the 1st demo submission deadline has been extended by 9 hours.
  - 2nd_demo deadline: 2026/04/08 (Wed.) 12:00:00
- Please upload the following files under 09_SUBMIT
  - **04_MEM_iclabXXX (Folder)**

    All memory-related files must be placed directly under this folder. Do not create any subdirectories (e.g., 04_MEM_iclabXXX/04_MEM) to store the files, or your lab will be considered failed.
  - **{your_cycle_time}_iclabXXX.txt**

    In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.
  - **DM_iclabXXX.v**
  - **filelist_iclabXXX.f**

    The memory path defined in filelist.f should be **related path** instead of absolute path.

No comment contexts are allowed in filelist.f

- If your files violate the naming rule, you will get 5 deduction points.

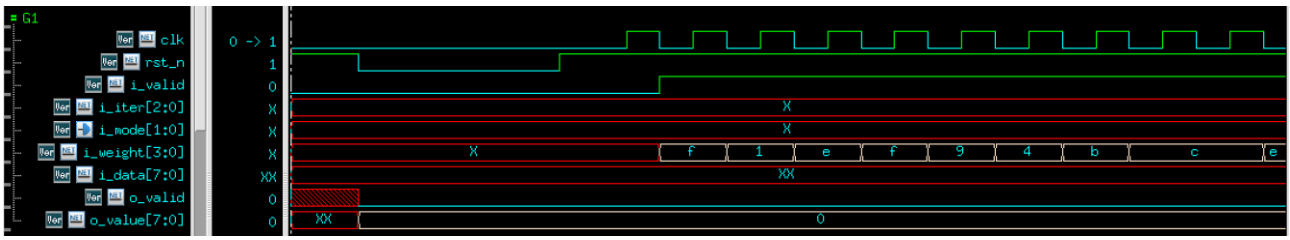
- Template folders and reference commands:
  - 01_RTL/ (RTL simulation) **./01_run_vcs_rtl**
  - 02_SYN/ (Synthesis) **./01_run_dc_shell**
    - Check if there is any latch in your design in syn.log
    - Check the timing of design in /Report/DM.timing
    - Check the area of design in /Report/DM.area
  - 03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**
  - 04_MEM/ (Memory location)
    - Your should generate your own memory and put the required files here
  - 09_SUBMIT/ (Submit your files) **./00_tar**

    ./01_submit

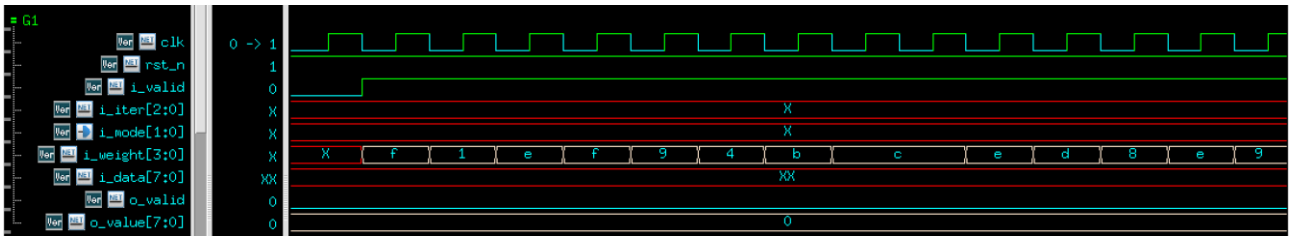
    ./02_check

Reference Waveform

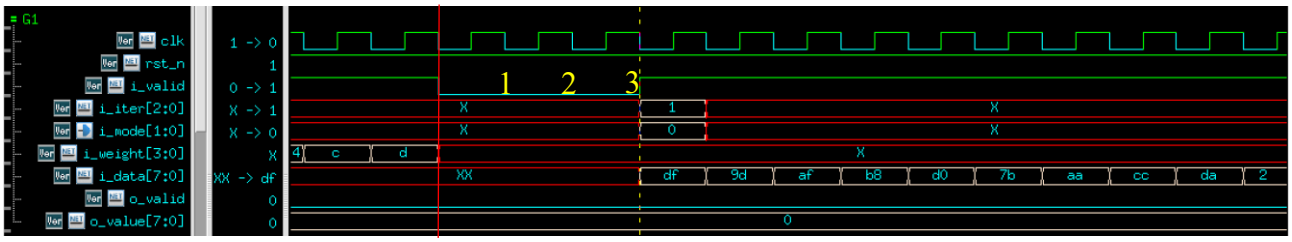
- Asynchronous reset and active-low reset all output to low



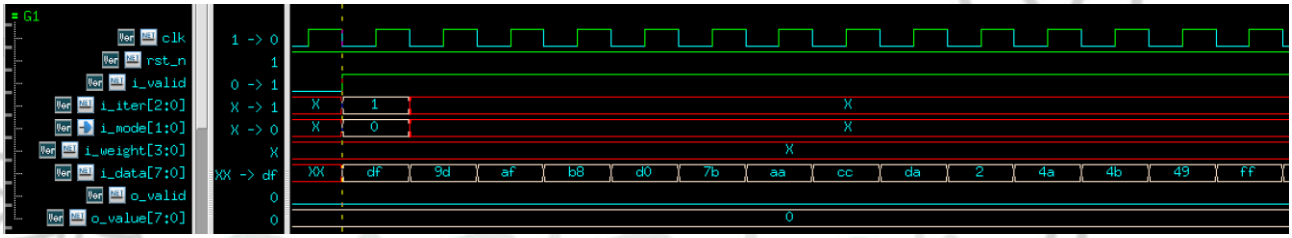
- Input of weight when i_valid is high(total1312 cycles)



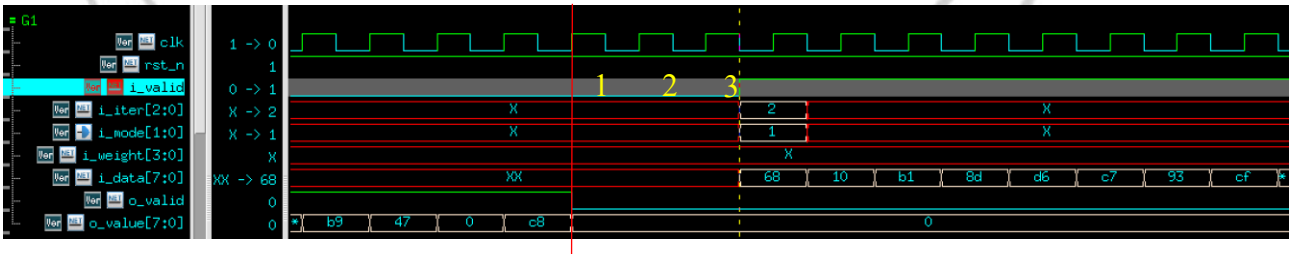
- i_valid will come in 1~3 negative edge of clock after weight is transmitted



- Input of image (total 64x64 cycles) with iteration and interpolation mode at 1st cycle



- The next input pattern will come in 1~3 negative edge of clock after the last o_valid of this pattern falls



- The definition of latency

